Chapter 7. Process Scheduling

Like every time sharing system, Linux achieves the magical effect of an apparent simultaneous execution of multiple processes by switching from one process to another in a very short time frame. Process switching itself was discussed in Chapter 3; this chapter deals with scheduling , which is concerned with when to switch and which process to choose.

The chapter consists of three parts. The section "Scheduling Policy" introduces the choices made by Linux in the abstract to schedule processes. The section "The Scheduling Algorithm" discusses the data structures used to implement scheduling and the corresponding algorithm. Finally, the section "System Calls Related to Scheduling" describes the system calls that affect process scheduling.

To simplify the description, we refer as usual to the 80 x 86 architecture; in particular, we assume that the system uses the Uniform Memory Access model, and that the system tick is set to 1 ms.

7.1. Scheduling Policy

The scheduling algorithm of traditional Unix operating systems must fulfill several conflicting objectives: fast process response time, good throughput for background jobs, avoidance of process starvation, reconciliation of the needs of low- and high-priority processes, and so on. The set of rules used to determine when and how to select a new process to run is called scheduling policy .

Linux scheduling is based on the time sharing technique: several processes run in "time multiplexing" because the CPU time is divided into slices, one for each runnable process.[*] Of course, a single processor can run only one process at any given instant. If a currently running process is not terminated when its time slice or quantum expires, a process switch may take place. Time sharing relies on timer interrupts and is thus transparent to processes. No additional code needs to be inserted in the programs to ensure CPU time sharing.

> [*] Recall that stopped and suspended processes cannot be selected by the scheduling algorithm to run on a CPU.

The scheduling policy is also based on ranking processes according to their priority. Complicated algorithms are sometimes used to derive the current priority of a process, but the end result is the same: each process is associated with a value that tells the scheduler how appropriate it is to let the process run on a CPU.

In Linux, process priority is dynamic. The scheduler keeps track of what processes are doing and adjusts their priorities periodically; in this way, processes that have been denied the use of a CPU for a long time interval are boosted by dynamically increasing their priority. Correspondingly, processes running for a long time are penalized by decreasing their priority.

When speaking about scheduling, processes are traditionally classified as I/O-bound or CPU-bound. The former make heavy use of I/O devices and spend much time waiting for I/O operations to complete; the latter carry on number-crunching applications that require a lot of CPU time.

An alternative classification distinguishes three classes of processes:

Interactive processes
: These interact constantly with their users, and therefore spend a lot of time waiting for keypresses and mouse operations. When input is received, the process must be woken up quickly, or the user will find the system to be unresponsive. Typically, the average delay must fall between 50 and 150 milliseconds. The variance of such delay must also be bounded, or the user will find the system to be erratic. Typical interactive programs are command shells, text editors, and graphical applications.

Batch processes
: These do not need user interaction, and hence they often run in the background. Because such processes do not need to be very responsive, they are often penalized by the scheduler. Typical batch programs are programming language compilers,

database search engines, and scientific computations.

Real-time processes
: These have very stringent scheduling requirements. Such processes should never be blocked by lower-priority processes and should have a short guaranteed response time with a minimum variance. Typical real-time programs are video and sound applications, robot controllers, and programs that collect data from physical sensors.

The two classifications we just offered are somewhat independent. For instance, a batch process can be either I/O-bound (e.g., a database server) or CPU-bound (e.g., an image-rendering program). While real-time programs are explicitly recognized as such by the scheduling algorithm in Linux, there is no easy way to distinguish between interactive and batch programs. The Linux 2.6 scheduler implements a sophisticated heuristic algorithm based on the past behavior of the processes to decide whether a given process should be considered as interactive or batch. Of course, the scheduler tends to favor interactive processes over batch ones.

Programmers may change the scheduling priorities by means of the system calls illustrated in Table 7-1. More details are given in the section "System Calls Related to Scheduling."

Table 7-1. System calls related to scheduling

System call	Description
`nice( )`	Change the static priority of a conventional process
`getpriority( )`	Get the maximum static priority of a group of conventional processes
`setpriority( )`	Set the static priority of a group of conventional processes
`sched_getscheduler( )`	Get the scheduling policy of a process
`sched_setscheduler( )`	Set the scheduling policy and the real-time priority of a process
`sched_getparam( )`	Get the real-time priority of a process
`sched_setparam( )`	Set the real-time

	priority of a process
`sched_yield( )`	Relinquish the processor voluntarily without blocking
`sched_get_ priority_min( )`	Get the minimum real-time priority value for a policy
`sched_get_ priority_max( )`	Get the maximum real-time priority value for a policy
`sched_rr_get_interval( )`	Get the time quantum value for the Round Robin policy
sched_setaffinity( )	Set the CPU affinity mask of a process
sched_getaffinity( )	Get the CPU affinity mask of a process

7.1.1. Process Preemption

As mentioned in the first chapter, Linux processes are preemptable. When a process enters the `TASK_RUNNING` state, the kernel checks whether its dynamic priority is greater than the priority of the currently running process. If it is, the execution of `current` is interrupted and the scheduler is invoked to select another process to run (usually the process that just became runnable). Of course, a process also may be preempted when its time quantum expires. When this occurs, the `TIF_NEED_RESCHED` flag in the `thread_info` structure of the current process is set, so the scheduler is invoked when the timer interrupt handler terminates.

For instance, let's consider a scenario in which only two programsa text editor and a compilerare being executed. The text editor is an interactive program, so it has a higher dynamic priority than the compiler. Nevertheless, it is often suspended, because the user alternates between pauses for think time and data entry; moreover, the average delay between two keypresses is relatively long. However, as soon as the user presses a key, an interrupt is raised and the kernel wakes up the text editor process. The kernel also determines that the dynamic priority of the editor is higher than the priority of `current`, the currently running process (the compiler), so it sets the `TIF_NEED_RESCHED` flag of this process, thus forcing the scheduler to be activated when the kernel finishes handling the interrupt. The scheduler selects the editor and performs a process switch; as a result, the execution of the editor is resumed very quickly and the character typed by the user is echoed to the screen. When the character has been processed, the text editor process suspends itself waiting for another keypress and the compiler process can resume its execution.

Be aware that a preempted process is not suspended, because it remains in the TASK_RUNNING state; it simply no longer uses the CPU. Moreover, remember that the Linux 2.6 kernel is preemptive, which means that a process can be preempted either when executing in Kernel or in User Mode; we discussed in depth this feature in the section "Kernel Preemption" in Chapter 5.

7.1.2. How Long Must a Quantum Last?

The quantum duration is critical for system performance: it should be neither too long nor too short.

If the average quantum duration is too short, the system overhead caused by process switches becomes excessively high. For instance, suppose that a process switch requires 5 milliseconds; if the quantum is also set to 5 milliseconds, then at least 50 percent of the CPU cycles will be dedicated to process switching.[*]

> [*] Actually, things could be much worse than this; for example, if the time required for the process switch is counted in the process quantum, all CPU time is devoted to the process switch and no process can progress toward its termination.

If the average quantum duration is too long, processes no longer appear to be executed concurrently. For instance, let's suppose that the quantum is set to five seconds; each runnable process makes progress for about five seconds, but then it stops for a very long time (typically, five seconds times the number of runnable processes).

It is often believed that a long quantum duration degrades the response time of interactive applications. This is usually false. As described in the section "Process Preemption" earlier in this chapter, interactive processes have a relatively high priority, so they quickly preempt the batch processes, no matter how long the quantum duration is.

In some cases, however, a very long quantum duration degrades the responsiveness of the system. For instance, suppose two users concurrently enter two commands at the respective shell prompts; one command starts a CPU-bound process, while the other launches an interactive application. Both shells fork a new process and delegate the execution of the user's command to it; moreover, suppose such new processes have the same initial priority (Linux does not know in advance if a program to be executed is batch or interactive). Now if the scheduler selects the CPU-bound process to run first, the other process could wait for a whole time quantum before starting its execution. Therefore, if the quantum duration is long, the system could appear to be unresponsive to the user that launched the interactive application.

The choice of the average quantum duration is always a compromise. The rule of thumb adopted by Linux is choose a duration as long as possible, while keeping good system response time.

7.2. The Scheduling Algorithm

The scheduling algorithm used in earlier versions of Linux was quite simple and straightforward: at every process switch the kernel scanned the list of runnable processes, computed their priorities, and selected the "best" process to run. The main drawback of that algorithm is that the time spent in choosing the best process depends on the number of runnable processes; therefore, the algorithm is too costlythat is, it spends too much timein high-end systems running thousands of processes.

The scheduling algorithm of Linux 2.6 is much more sophisticated. By design, it scales well with the number of runnable processes, because it selects the process to run in constant time, independently of the number of runnable processes. It also scales well with the number of processors because each CPU has its own queue of runnable processes. Furthermore, the new algorithm does a better job of distinguishing interactive processes and batch processes. As a consequence, users of heavily loaded systems feel that interactive applications are much more responsive in Linux 2.6 than in earlier versions.

The scheduler always succeeds in finding a process to be executed; in fact, there is always at least one runnable process: the swapper process, which has PID 0 and executes only when the CPU cannot execute other processes. As mentioned in Chapter 3, every CPU of a multiprocessor system has its own swapper process with PID equal to 0.

Every Linux process is always scheduled according to one of the following scheduling classes :

SCHED_FIFO
: A First-In, First-Out real-time process. When the scheduler assigns the CPU to the process, it leaves the process descriptor in its current position in the runqueue list. If no other higher-priority real-time process is runnable, the process continues to use the CPU as long as it wishes, even if other real-time processes that have the same priority are runnable.

SCHED_RR
: A Round Robin real-time process. When the scheduler assigns the CPU to the process, it puts the process descriptor at the end of the runqueue list. This policy ensures a fair assignment of CPU time to all `SCHED_RR` real-time processes that have the same priority.

SCHED_NORMAL
: A conventional, time-shared process.

The scheduling algorithm behaves quite differently depending on whether the process is conventional or real-time.

7.2.1. Scheduling of Conventional Processes

Every conventional process has its own static priority, which is a value used by the scheduler to rate the process with respect to the other conventional processes in the system. The kernel represents the static priority of a conventional process with a number ranging from 100 (highest priority) to 139 (lowest priority); notice that static priority decreases as the values increase.

A new process always inherits the static priority of its parent. However, a user can change the static priority of the processes that he owns by passing some "nice values" to the `nice( )` and `setpriority( )` system calls (see the section "System Calls Related to Scheduling" later in this chapter).

7.2.1.1. Base time quantum

The static priority essentially determines the base time quantum of a process, that is, the time quantum duration assigned to the process when it has exhausted its previous time quantum. Static priority and base time quantum are related by the following formula:

$$\underset{\text{(in milliseconds)}}{\textit{base time quantum}} = \begin{cases} (140 - \textit{static priority}) \times 20 & \text{if } \textit{static priority} < 120 \\ (140 - \textit{static priority}) \times 5 & \text{if } \textit{static priority} \geq 120 \end{cases} \quad (1)$$

As you see, the higher the static priority (i.e., the lower its numerical value), the longer the base time quantum. As a consequence, higher priority processes usually get longer slices of CPU time with respect to lower priority processes. Table 7-2 shows the static priority, the base time quantum values, and the corresponding nice values for a conventional process having highest static priority, default static priority, and lowest static priority. (The table also lists the values of the interactive delta and of the sleep time threshold, which are explained later in this chapter.)

Table 7-2. Typical priority values for a conventional process

Description	Static priority	Nice value	Base time quantum	Interactivedelta	Sle tim thr
Highest static priority	100	-20	800 ms	-3	29
High static priority	110	−10	600 ms	−1	49
Default static priority	120	0	100 ms	+2	79
Low static priority	130	+10	50 ms	+4	99
Lowest static priority	139	+19	5 ms	+6	11

7.2.1.2. Dynamic priority and average sleep time

Besides a static priority, a conventional process also has a dynamic priority, which is a value ranging from 100 (highest priority) to 139 (lowest priority). The dynamic priority is the number actually looked up by the scheduler when selecting the new process to run. It is related to the static priority by the following empirical formula:

```
dynamic priority = max (100, min (  static priority - bonus
 + 5, 139))   (2)
```

The bonus is a value ranging from 0 to 10; a value less than 5 represents a penalty that lowers the dynamic priority, while a value greater than 5 is a premium that raises the dynamic priority. The value of the bonus, in turn, depends on the past history of the process; more precisely, it is related to the average sleep time of the process.

Roughly, the average sleep time is the average number of nanoseconds that the process spent while sleeping. Be warned, however, that this is not an average operation on the elapsed time. For instance, sleeping in `TASK_INTERRUPTIBLE` state contributes to the average sleep time in a different way from sleeping in `TASK_UNINTERRUPTIBLE` state. Moreover, the average sleep time decreases while a process is running. Finally, the average sleep time can never become larger than 1 second.

The correspondence between average sleep times and bonus values is shown in Table 7-3. (The table lists also the corresponding granularity of the time slice, which will be discussed later.)

Table 7-3. Average sleep times, bonus values, and time slice granularity

Average sleep time	Bonus	Granularity
Greater than or equal to 0 but smaller than 100 ms	0	5120
Greater than or equal to 100 ms but smaller than 200 ms	1	2560
Greater than or equal to 200 ms but smaller than 300 ms	2	1280
Greater than or equal to 300 ms but smaller than 400 ms		

4

3

640

Greater than or equal to 400 ms but smaller than 500 ms

4

320

Greater than or equal to 500 ms but smaller than 600 ms

5

160

Greater than or equal to 600 ms but smaller than 700 ms

6

80

Greater than or equal to 700 ms but smaller than 800 ms

7

40

Greater than or equal to 800 ms but smaller than 900 ms

8

20

Greater than or equal to 900 ms but smaller than 1000 ms

9

10

1 second

10

10

The average sleep time is also used by the scheduler to determine whether a given process should be considered interactive or batch. More precisely, a process is considered "interactive" if it satisfies the following formula:

```
dynamic priority  ≤  3 x   static priority / 4 + 28       (3)
```

which is equivalent to the following:

```
bonus - 5  ≥  static priority / 4 - 28
```

The expression static priority / 4 - 28 is called the interactive delta ; some typical values of this term are listed in Table 7-2. It should be noted that it is far easier for high priority than for low priority processes to become interactive. For instance, a process having highest static priority (100) is considered interactive when its bonus value exceeds 2, that is, when its average sleep time exceeds 200 ms. Conversely, a process having lowest static priority (139) is never considered as interactive, because the bonus value is always smaller than the value 11 required to reach an interactive delta equal to 6. A process having default static priority (120) becomes interactive as soon as its average sleep time exceeds 700 ms.

7.2.1.3. Active and expired processes

Even if conventional processes having higher static priorities get larger slices of the CPU time, they should not completely lock out the processes having lower static priority. To avoid process starvation, when a process finishes its time quantum, it can be replaced by a lower priority process whose time quantum has not yet been exhausted. To implement this mechanism, the scheduler keeps two disjoint sets of runnable processes:

Active processes
: These runnable processes have not yet exhausted their time quantum and are thus allowed to run.

Expired processes
: These runnable processes have exhausted their time quantum and are thus forbidden to run until all active processes expire.

However, the general schema is slightly more complicated than this, because the scheduler tries to boost the performance of interactive processes. An active batch process that finishes its time quantum always becomes expired. An active interactive process that finishes its time quantum usually remains active: the scheduler refills its time quantum and leaves it in the set of active processes. However, the scheduler moves an interactive process that finished its time quantum into the set of expired processes if the eldest expired process has already waited for a long time, or if an expired process has higher static priority (lower value) than the interactive process. As a consequence, the set of active processes will eventually become empty and the expired processes will have a chance to run.

7.2.2. Scheduling of Real-Time Processes

Every real-time process is associated with a real-time priority, which is a value ranging from 1 (highest priority) to 99 (lowest priority). The scheduler always favors a higher priority runnable process over a lower priority one; in other words, a real-time process inhibits the execution of every lower-priority process while it remains runnable. Contrary to conventional processes, real-time processes are always considered active (see the previous section). The user can change the real-time priority of a process by means of the `sched_setparam( )` and `sched_setscheduler( )` system calls (see the section "System Calls

Related to Scheduling" later in this chapter).

If several real-time runnable processes have the same highest priority, the scheduler chooses the process that occurs first in the corresponding list of the local CPU's runqueue (see the section "The lists of TASK_RUNNING processes" in Chapter 3).

A real-time process is replaced by another process only when one of the following events occurs:

- The process is preempted by another process having higher real-time priority.
- The process performs a blocking operation, and it is put to sleep (in state `TASK_INTERRUPTIBLE` or `TASK_UNINTERRUPTIBLE`).
- The process is stopped (in state `TASK_STOPPED` or `TASK_TRACED`), or it is killed (in state `EXIT_ZOMBIE` or `EXIT_DEAD`).
- The process voluntarily relinquishes the CPU by invoking the `sched_yield( )` system call (see the section "System Calls Related to Scheduling" later in this chapter).
- The process is Round Robin real-time (`SCHED_RR`), and it has exhausted its time quantum.

The `nice( )` and `setpriority( )` system calls, when applied to a Round Robin real-time process, do not change the real-time priority but rather the duration of the base time quantum. In fact, the duration of the base time quantum of Round Robin real-time processes does not depend on the real-time priority, but rather on the static priority of the process, according to the formula (1) in the earlier section "Scheduling of Conventional Processes."

7.3. Data Structures Used by the Scheduler

Recall from the section "Identifying a Process" in Chapter 3 that the process list links all process descriptors, while the runqueue lists link the process descriptors of all runnable processesthat is, of those in a TASK_RUNNING stateexcept the swapper process (idle process).

7.3.1. The runqueue Data Structure

The runqueue data structure is the most important data structure of the Linux 2.6 scheduler. Each CPU in the system has its own runqueue; all runqueue structures are stored in the runqueues per-CPU variable (see the section "Per-CPU Variables" in Chapter 5). The this_rq( ) macro yields the address of the runqueue of the local CPU, while the cpu_rq(n) macro yields the address of the runqueue of the CPU having index n.

Table 7-4 lists the fields included in the runqueue data structure; we will discuss most of them in the following sections of the chapter.

Table 7-4. The fields of the runqueue structure

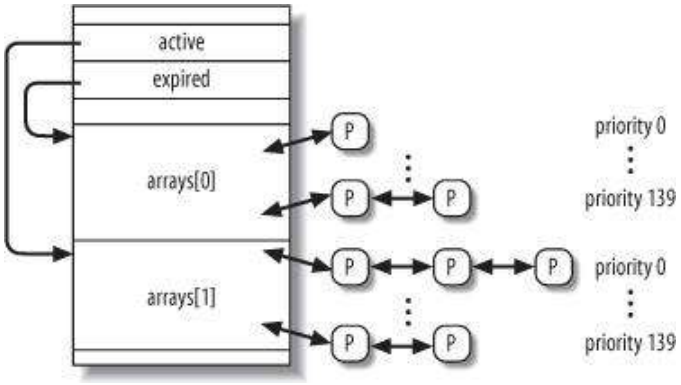
Type	Name	Description
spinlock_t	lock	Spin lock protecting the lists of processes
unsigned long	nr_running	Number of runnable processes in the runqueue lists
unsigned long	cpu_load	CPU load factor based on the average number of processes in the runqueue
unsigned long	nr_switches	Number of process switches performed by the CPU
unsigned long	nr_uninterruptible	Number of processes that were previously in the runqueue lists and are now sleeping in TASK_UNINTERRUPTIBLE state (only the sum of these fields across all runqueues is meaningful)
unsigned long	expired_timestamp	

		Insertion time of the eldest process in the expired lists
unsigned long long	timestamp_last_tick	Timestamp value of the last timer interrupt
task_t *	`curr`	Process descriptor pointer of the currently running process (same as `current` for the local CPU)
task_t *	`idle`	Process descriptor pointer of the swapper process for this CPU
struct mm_struct *	`prev_mm`	Used during a process switch to store the address of the memory descriptor of the process being replaced
prio_array_t *	`active`	Pointer to the lists of active processes
prio_array_t *	`expired`	Pointer to the lists of expired processes
prio_array_t [2]	`arrays`	The two sets of active and expired processes
int	best_expired_prio	The best static priority (lowest value) among the expired processes
atomic_t	`nr_iowait`	Number of processes that were previously in the runqueue lists and are now waiting for a disk I/O operation to complete
struct sched_domain *	sd	Points to the base scheduling domain of this CPU (see the section "Scheduling Domains" later in this chapter)
int	`active_balance`	Flag set if some process shall be migrated from this runqueue to another (runqueue balancing)
int	`push_cpu`	Not used
task_t *	`migration_thread`	Process descriptor pointer of the migration kernel thread
struct list_head	`migration_queue`	List of processes to be removed from the runqueue

The most important fields of the `runqueue` data structure are those related to the lists of runnable processes. Every runnable process in the system belongs to one, and just one, runqueue. As long as a runnable process remains in the same runqueue, it can be executed only by the CPU owning that runqueue. However, as we'll see later, runnable processes may migrate from one runqueue to another.

The `arrays` field of the runqueue is an array consisting of two `prio_array_t` structures. Each data structure represents a set of runnable processes, and includes 140 doubly linked list heads (one list for each possible process priority), a priority bitmap, and a counter of the processes included in the set (see Table 3-2 in the section Chapter 3).

Figure 7-1. The runqueue structure and the two sets of runnable processes



As shown in Figure 7-1, the `active` field of the `runqueue` structure points to one of the two `prio_array_t` data structures in `arrays`: the corresponding set of runnable processes includes the active processes. Conversely, the `expired` field points to the other `prio_array_t` data structure in `arrays`: the corresponding set of runnable processes includes the expired processes.

Periodically, the role of the two data structures in `arrays` changes: the active processes suddenly become the expired processes, and the expired processes become the active ones. To achieve this change, the scheduler simply exchanges the contents of the `active` and `expired` fields of the runqueue.

7.3.2. Process Descriptor

Each process descriptor includes several fields related to scheduling; they are listed in Table 7-5.

Table 7-5. Fields of the process descriptor related to the scheduler

Type

Name

Description

unsigned long

thread_info->flags

Stores the `TIF_NEED_RESCHED` flag, which is set if the scheduler must be invoked (see the section "Returning from Interrupts and Exceptions" in Chapter 4)

unsigned int

thread_info->cpu

Logical number of the CPU owning the runqueue to which the runnable process belongs

unsigned long

state

The current state of the process (see the section "Process State" in Chapter 3)

`int`

`prio`

Dynamic priority of the process

`int`

`static_prio`

Static priority of the process

`struct list_head`

`run_list`

Pointers to the next and previous elements in the runqueue list to which the process belongs

`prio_array_t *`

`array`

Pointer to the runqueue's `prio_array_t` set that includes the process

`unsigned long`

`sleep_avg`

Average sleep time of the process

`unsigned long long`

`timestamp`

Time of last insertion of the process in the runqueue, or time of last process switch involving the process

`unsigned long long`

`last_ran`

Time of last process switch that replaced the process

`int`

`activated`

Condition code used when the process is awakened

`unsigned long`

`policy`

The scheduling class of the process (SCHED_NORMAL, SCHED_RR, or SCHED_FIFO)

cpumask_t

cpus_allowed

Bit mask of the CPUs that can execute the process

unsigned int

time_slice

Ticks left in the time quantum of the process

unsigned int

first_time_slice

Flag set to 1 if the process never exhausted its time quantum

unsigned long

rt_priority

Real-time priority of the process

When a new process is created, sched_fork( ), invoked by copy_process( ), sets the time_slice field of both current (the parent) and p (the child) processes in the following way:

```
p->time_slice = (current->time_slice + 1) >> 1;
current->time_slice >>= 1;
```

In other words, the number of ticks left to the parent is split in two halves: one for the parent and one for the child. This is done to prevent users from getting an unlimited amount of CPU time by using the following method: the parent process creates a child process that runs the same code and then kills itself; by properly adjusting the creation rate, the child process would always get a fresh quantum before the quantum of its parent expires. This programming trick does not work because the kernel does not reward forks. Similarly, a user cannot hog an unfair share of the processor by starting several background processes in a shell or by opening a lot of windows on a graphical desktop. More generally speaking, a process cannot hog resources (unless it has privileges to give itself a real-time policy) by forking multiple descendents.

If the parent had just one tick left in its time slice, the splitting operation forces current->time_slice to 0, thus exhausting the quantum of the parent. In this case, copy_process( ) sets current->time_slice back to 1, then invokes scheduler_tick( ) to decrease the field (see the following section).

The copy_process( ) function also initializes a few other fields of the child's process descriptor related to scheduling:

```
p->first_time_slice = 1;
p->timestamp = sched_clock( );
```

The `first_time_slice` flag is set to 1, because the child has never exhausted its time quantum (if a process terminates or executes a new program during its first time slice, the parent process is rewarded with the remaining time slice of the child). The `timestamp` field is initialized with a timestamp value produced by `sched_clock( )`: essentially, this function returns the contents of the 64-bit TSC register (see the section "Time Stamp Counter (TSC)" in Chapter 6) converted to nanoseconds.

7.4. Functions Used by the Scheduler

The scheduler relies on several functions in order to do its work; the most important are:

scheduler_tick( )
: Keeps the `time_slice` counter of `current` up-to-date

try_to_wake_up( )
: Awakens a sleeping process

recalc_task_prio( )
: Updates the dynamic priority of a process

schedule( )
: Selects a new process to be executed

load_balance()
: Keeps the runqueues of a multiprocessor system balanced

7.4.1. The scheduler_tick( ) Function

We have already explained in the section "Updating Local CPU Statistics" in Chapter 6 how `scheduler_tick( )` is invoked once every tick to perform some operations related to scheduling. It executes the following main steps:

1. Stores in the `timestamp_last_tick` field of the local runqueue the current value of the TSC converted to nanoseconds; this timestamp is obtained from the `sched_clock( )` function (see the previous section).
2. Checks whether the current process is the swapper process of the local CPU. If so, it performs the following substeps:

    a. If the local runqueue includes another runnable process besides swapper, it sets the `TIF_NEED_RESCHED` flag of the current process to force rescheduling. As we'll see in the section "The schedule( ) Function" later in this chapter, if the kernel supports the hyper-threading technology (see the section "Runqueue Balancing in Multiprocessor Systems" later in this chapter), a logical CPU might be idle even if there are runnable processes in its runqueue, as long as those processes have significantly lower priorities than the priority of a process already executing on another logical CPU associated with the same physical CPU.

   b. Jumps to step 7 (there is no need to update the time slice counter of the swapper process).
3. Checks whether `current->array` points to the active list of the local runqueue. If not, the process has expired its time quantum, but it has not yet been replaced: sets the `TIF_NEED_RESCHED` flag to force rescheduling, and jumps to step 7.
4. Acquires the `this_rq()->lock` spin lock.
5. Decreases the time slice counter of the current process, and checks whether the quantum is exhausted. The operations performed by the function are quite different according to the scheduling class of the process; we will discuss them in a moment.
6. Releases the `this_rq( )->lock` spin lock.
7. Invokes the `rebalance_tick( )` function, which should ensure that the runqueues of the various CPUs contain approximately the same number of runnable processes. We will discuss runqueue balancing in the later section "Runqueue Balancing in Multiprocessor Systems."

7.4.1.1. Updating the time slice of a real-time process

If the current process is a FIFO real-time process, `scheduler_tick( )` has nothing to do. In this case, in fact, `current` cannot be preempted by lower or equal priority processes, thus it does not make sense to keep its time slice counter up-to-date.

If `current` is a Round Robin real-time process, `scheduler_tick( )` decreases its time slice counter and checks whether the quantum is exhausted:

```
if (current->policy == SCHED_RR && !--current->time_slice) {
    current->time_slice = task_timeslice(current);
    current->first_time_slice = 0;
    set_tsk_need_resched(current);
    list_del(&current->run_list);
    list_add_tail(&current->run_list,
                  this_rq( )->active->queue+current->prio);
}
```

If the function determines that the time quantum is effectively exhausted, it performs a few operations aimed to ensure that `current` will be preempted, if necessary, as soon as possible.

The first operation consists of refilling the time slice counter of the process by invoking `task_timeslice( )`. This function considers the static priority of the process and returns the corresponding base time quantum, according to the formula (1) shown in the earlier section "Scheduling of Conventional Processes." Moreover, the `first_time_slice` field of `current` is cleared: this flag is set by `copy_process( )` in the service routine of the `fork( )` system call, and should be cleared as soon as the first time quantum of the process elapses.

Next, `scheduler_tick( )` invokes the `set_tsk_need_resched( )` function to set the `TIF_NEED_RESCHED` flag of the process. As described in the section "Returning from Interrupts and Exceptions" in Chapter 4, this flag forces

the invocation of the `schedule( )` function, so that `current` can be replaced by another real-time process having equal (or higher) priority, if any.

The last operation of `scheduler_tick( )` consists of moving the process descriptor to the last position of the runqueue active list corresponding to the priority of `current`. Placing `current` in the last position ensures that it will not be selected again for execution until every real-time runnable process having the same priority as `current` will get a slice of the CPU time. This is the meaning of round-robin scheduling. The descriptor is moved by first invoking `list_del( )` to remove the process from the runqueue active list, then by invoking `list_add_tail( )` to insert back the process in the last position of the same list.

7.4.1.2. Updating the time slice of a conventional process

If the current process is a conventional process, the `scheduler_tick( )` function performs the following operations:

1. Decreases the time slice counter (`current->time_slice`).
2. Checks the time slice counter. If the time quantum is exhausted, the function performs the following operations:

    a. Invokes `dequeue_task( )` to remove `current` from the `this_rq( )->active` set of runnable processes.
    b. Invokes `set_tsk_need_resched( )` to set the `TIF_NEED_RESCHED` flag.
    c. Updates the dynamic priority of `current`:

    ```
    current->prio = effective_prio(current);
    ```

    The `effective_prio( )` function reads the `static_prio` and `sleep_avg` fields of `current`, and computes the dynamic priority of the process according to the formula (2) shown in the earlier section "Scheduling of Conventional Processes."
    d. Refills the time quantum of the process:

    ```
    current->time_slice = task_timeslice(current);
    current->first_time_slice = 0;
    ```

    e. If the `expired_timestamp` field of the local runqueue data structure is equal to zero (that is, the set of expired processes is empty), writes into the field the value of the current tick:

    ```
    if (!this_rq( )->expired_timestamp)
        this_rq( )->expired_timestamp = jiffies;
    ```

    f. Inserts the current process either in the active set or in the expired set:

```
if (!TASK_INTERACTIVE(current) || EXPIRED_STARVING(this_rq( )) {
    enqueue_task(current, this_rq( )->expired);
    if (current->static_prio < this_rq( )->best_expired_prio)
        this_rq( )->best_expired_prio = current->static_prio;
} else
    enqueue_task(current, this_rq( )->active);
```

The TASK_INTERACTIVE macro yields the value one if the process is recognized as interactive using the formula (3) shown in the earlier section "Scheduling of Conventional Processes." The EXPIRED_STARVING macro checks whether the first expired process in the runqueue had to wait for more than 1000 ticks times the number of runnable processes in the runqueue plus one; if so, the macro yields the value one. The EXPIRED_STARVING macro also yields the value one if the static priority value of the current process is greater than the static priority value of an already expired process.

3. Otherwise, if the time quantum is not exhausted (current->time_slice is not zero), checks whether the remaining time slice of the current process is too long:

```
if (TASK_INTERACTIVE(p) && !((task_timeslice(p) -
        p->time_slice) % TIMESLICE_GRANULARITY(p)) &&
        (p->time_slice >= TIMESLICE_GRANULARITY(p)) &&
        (p->array == rq->active)) {
    list_del(&current->run_list);
    list_add_tail(&current->run_list,
                this_rq( )->active->queue+current->prio);
    set_tsk_need_resched(p);
}
```

The TIMESLICE_GRANULARITY macro yields the product of the number of CPUs in the system and a constant proportional to the bonus of the current process (see Table 7-3 earlier in the chapter). Basically, the time quantum of interactive processes with high static priorities is split into several pieces of TIMESLICE_GRANULARITY size, so that they do not monopolize the CPU.

7.4.2. The try_to_wake_up( ) Function

The TRy_to_wake_up( ) function awakes a sleeping or stopped process by setting its state to TASK_RUNNING and inserting it into the runqueue of the local CPU. For instance, the function is invoked to wake up processes included in a wait queue (see the section "How Processes Are Organized" in Chapter 3) or to resume execution of processes waiting for a signal (see Chapter 11). The function receives as its parameters:

- The descriptor pointer (p) of the process to be awakened
- A mask of the process states (state) that can be awakened

- A flag (`sync`) that forbids the awakened process to preempt the process currently running on the local CPU

The function performs the following operations:

1. Invokes the `task_rq_lock( )` function to disable local interrupts and to acquire the lock of the runqueue `rq` owned by the CPU that was last executing the process (it could be different from the local CPU). The logical number of that CPU is stored in the `p->thread_info->cpu` field.
2. Checks if the state of the process `p->state` belongs to the mask of states `state` passed as argument to the function; if this is not the case, it jumps to step 9 to terminate the function.
3. If the `p->array` field is not `NULL`, the process already belongs to a runqueue; therefore, it jumps to step 8.
4. In multiprocessor systems, it checks whether the process to be awakened should be migrated from the runqueue of the lastly executing CPU to the runqueue of another CPU. Essentially, the function selects a target runqueue according to some heuristic rules. For example:

   - If some CPU in the system is idle, it selects its runqueue as the target. Preference is given to the previously executing CPU and to the local CPU, in this order.
   - If the workload of the previously executing CPU is significantly lower than the workload of the local CPU, it selects the old runqueue as the target.
   - If the process has been executed recently, it selects the old runqueue as the target (the hardware cache might still be filled with the data of the process).
   - If moving the process to the local CPU reduces the unbalance between the CPUs, the target is the local runqueue (see the section "Runqueue Balancing in Multiprocessor Systems" later in this chapter).

   After this step has been executed, the function has identified a target CPU that will execute the awakened process and, correspondingly, a target runqueue `rq` in which to insert the process.

1. If the process is in the `TASK_UNINTERRUPTIBLE` state, it decreases the `nr_uninterruptible` field of the target runqueue, and sets the `p->activated` field of the process descriptor to −1. See the later section "The recalc_task_prio( ) Function" for an explanation of the `activated` field.
2. Invokes the `activate_task( )` function, which in turn performs the following substeps:

   a. Invokes `sched_clock( )` to get the current timestamp in nanoseconds. If the target CPU is not the local CPU, it compensates for the drift of the local timer interrupts by using the timestamps relative to the last occurrences of the timer interrupts on the local and target CPUs:

```
now = (sched_clock( ) - this_rq( )->timestamp_last_tick)
      + rq->timestamp_last_tick;
```

b. Invokes `recalc_task_prio( )`, passing to it the process descriptor pointer and the timestamp computed in the previous step. The `recalc_task_prio( )` function is described in the next section.
c. Sets the value of the `p->activated` field according to Table 7-6 later in this chapter.
d. Sets the `p->timestamp` field with the timestamp computed in step 6a.
e. Inserts the process descriptor in the active set:

```
enqueue_task(p, rq->active);
rq->nr_running++;
```

3. If either the target CPU is not the local CPU or if the `sync` flag is not set, it checks whether the new runnable process has a dynamic priority higher than that of the current process of the `rq` runqueue (`p->prio < rq->curr->prio`); if so, invokes `resched_task( )` to preempt `rq->curr`. In uniprocessor systems the latter function just executes `set_tsk_need_resched( )` to set the `TIF_NEED_RESCHED` flag of the `rq->curr` process. In multiprocessor systems `resched_task( )` also checks whether the old value of whether `TIF_NEED_RESCHED` flag was zero, the target CPU is different from the local CPU, and whether the `TIF_POLLING_NRFLAG` flag of the `rq->curr` process is clear (the target CPU is not actively polling the status of the `TIF_NEED_RESCHED` flag of the process). If so, `resched_task( )` invokes `smp_send_reschedule( )` to raise an IPI and force rescheduling on the target CPU (see the section "Interprocessor Interrupt Handling" in Chapter 4).
4. Sets the `p->state` field of the process to `TASK_RUNNING`.
5. Invokes `task_rq_unlock( )` to unlock the `rq` runqueue and reenable the local interrupts.
6. Returns 1 (if the process has been successfully awakened) or 0 (if the process has not been awakened).

7.4.3. The recalc_task_prio( ) Function

The `recalc_task_prio( )` function updates the average sleep time and the dynamic priority of a process. It receives as its parameters a process descriptor pointer `p` and a timestamp `now` computed by the `sched_clock( )` function.

The function executes the following operations:

1. Stores in the `sleep_time` local variable the result of:

```
min (now    -   p->timestamp, 10⁹ )
```

The `p->timestamp` field contains the timestamp of the process switch that put the process to sleep; therefore, `sleep_time` stores the number of nanoseconds that the process spent sleeping since its last execution (or the

equivalent of 1 second, if the process slept more).

2. If `sleep_time` is not greater than zero, it jumps to step 8 so as to skip updating the average sleep time of the process.
3. Checks whether the process is not a kernel thread, whether it is awakening from the `TASK_UNINTERRUPTIBLE` state (`p->activated` field equal to -1; see step 5 in the previous section), and whether it has been continuously asleep beyond a given sleep time threshold. If these three conditions are fulfilled, the function sets the `p->sleep_avg` field to the equivalent of 900 ticks (an empirical value obtained by subtracting the duration of the base time quantum of a standard process from the maximum average sleep time). Then, it jumps to step 8.

   The sleep time threshold depends on the static priority of the process; some typical values are shown in Table 7-2. In short, the goal of this empirical rule is to ensure that processes that have been asleep for a long time in uninterruptible modeusually waiting for disk I/O operationsget a predefined sleep average value that is large enough to allow them to be quickly serviced, but it is also not so large to cause starvation for other processes.
4. Executes the `CURRENT_BONUS` macro to compute the bonus value of the previous average sleep time of the process (see Table 7-3). If (10 − bonus) is greater than zero, the function multiplies `sleep_time` by this value. Since `sleep_time` will be added to the average sleep time of the process (see step 6 below), the lower the current average sleep time is, the more rapidly it will rise.
5. If the process is in `TASK_UNINTERRUPTIBLE` mode and it is not a kernel thread, it performs the following substeps:

   a. Checks whether the average sleep time `p->sleep_avg` is greater than or equal to its sleep time threshold (see Table 7-2 earlier in this chapter). If so, it resets the `sleep_avg` local variable to zerothus skipping the adjustment of the average sleep timeand jumps to step 6.
   b. If the sum `sleep_avg + p->sleep_avg` is greater than or equal to the sleep time threshold, it sets the `p->sleep_avg` field to the sleep time threshold, and sets `sleep_avg` to zero.

   By somewhat limiting the increment of the average sleep time of the process, the function does not reward too much batch processes that sleep for a long time.
6. Adds `sleep_time` to the average sleep time of the process (`p->sleep_avg`).
7. Checks whether `p->sleep_avg` exceeds 1000 ticks (in nanoseconds); if so, the function cuts it down to 1000 ticks (in nanoseconds).
8. Updates the dynamic priority of the process:

```
p->prio = effective_prio(p);
```

The `effective_prio( )` function has already been discussed in the section "The scheduler_tick( ) Function" earlier in this chapter.

7.4.4. The schedule( ) Function

The `schedule( )` function implements the scheduler. Its objective is to find a process in the runqueue list and then assign the CPU to it. It is invoked, directly or in a lazy (deferred) way, by several kernel routines.

7.4.4.1. Direct invocation

The scheduler is invoked directly when the `current` process must be blocked right away because the resource it needs is not available. In this case, the kernel routine that wants to block it proceeds as follows:

1. Inserts `current` in the proper wait queue.
2. Changes the state of `current` either to `TASK_INTERRUPTIBLE` or to `TASK_UNINTERRUPTIBLE`.
3. Invokes `schedule( )`.
4. Checks whether the resource is available; if not, goes to step 2.
5. Once the resource is available, removes `current` from the wait queue.

The kernel routine checks repeatedly whether the resource needed by the process is available; if not, it yields the CPU to some other process by invoking `schedule( )`. Later, when the scheduler once again grants the CPU to the process, the availability of the resource is rechecked. These steps are similar to those performed by `wait_event( )` and similar functions described in the section "How Processes Are Organized" in Chapter 3.

The scheduler is also directly invoked by many device drivers that execute long iterative tasks. At each iteration cycle, the driver checks the value of the `TIF_NEED_RESCHED` flag and, if necessary, invokes `schedule( )` to voluntarily relinquish the CPU.

7.4.4.2. Lazy invocation

The scheduler can also be invoked in a lazy way by setting the `TIF_NEED_RESCHED` flag of `current` to 1. Because a check on the value of this flag is always made before resuming the execution of a User Mode process (see the section "Returning from Interrupts and Exceptions" in Chapter 4), `schedule( )` will definitely be invoked at some time in the near future.

Typical examples of lazy invocation of the scheduler are:

- When `current` has used up its quantum of CPU time; this is done by the `scheduler_tick( )` function.
- When a process is woken up and its priority is higher than that of the current process; this task is performed by the `try_to_wake_up( )` function.
- When a `sched_setscheduler( )` system call is issued (see the section "System Calls Related to Scheduling" later in this chapter).

7.4.4.3. Actions performed by schedule( ) before a process switch

The goal of the `schedule( )` function consists of replacing the currently executing process with another one. Thus, the key outcome of the function is to set a local variable called `next`, so that it points to the descriptor of the process selected to replace `current`. If no runnable process in the system has priority greater than the priority of `current`, at the end, `next` coincides with `current` and no process switch takes place.

The `schedule( )` function starts by disabling kernel preemption and initializing a few local variables:

```
need_resched:

preempt_disable( );
prev = current;
rq = this_rq( );
```

As you see, the pointer returned by `current` is saved in `prev`, and the address of the runqueue data structure corresponding to the local CPU is saved in `rq`.

Next, `schedule( )` makes sure that `prev` doesn't hold the big kernel lock (see the section "The Big Kernel Lock" in Chapter 5):

```
if (prev->lock_depth >= 0)
    up(&kernel_sem);
```

Notice that `schedule( )` doesn't change the value of the `lock_depth` field; when `prev` resumes its execution, it reacquires the `kernel_flag` spin lock if the value of this field is not negative. Thus, the big kernel lock is automatically released and reacquired across a process switch.

The `sched_clock( )` function is invoked to read the TSC and convert its value to nanoseconds; the timestamp obtained is saved in the `now` local variable. Then, `schedule( )` computes the duration of the CPU time slice used by `prev`:

```
now = sched_clock( );
run_time = now - prev->timestamp;
if (run_time > 1000000000)
    run_time = 1000000000;
```

The usual cut-off at 1 second (converted to nanoseconds) applies. The `run_time` value is used to charge the process for the CPU usage. However, a process having a high average sleep time is favored:

```
run_time /= (CURRENT_BONUS(prev) ? : 1);
```

Remember that CURRENT_BONUS returns a value between 0 and 10 that is proportional to the average sleep time of the process.

Before starting to look at the runnable processes, schedule( ) must disable the local interrupts and acquire the spin lock that protects the runqueue:

```
spin_lock_irq(&rq->lock);
```

As explained in the section "Process Termination" in Chapter 3, prev might be a process that is being terminated. To recognize this case, schedule( ) looks at the PF_DEAD flag:

```
if (prev->flags & PF_DEAD)
    prev->state = EXIT_DEAD;
```

Next, schedule( ) examines the state of prev. If it is not runnable and it has not been preempted in Kernel Mode (see the section "Returning from Interrupts and Exceptions" in Chapter 4), then it should be removed from the runqueue. However, if it has nonblocked pending signals and its state is TASK_INTERRUPTIBLE, the function sets the process state to TASK_RUNNING and leaves it into the runqueue. This action is not the same as assigning the processor to prev; it just gives prev a chance to be selected for execution:

```
if (prev->state != TASK_RUNNING &&
    !(preempt_count() & PREEMPT_ACTIVE)) {
    if (prev->state == TASK_INTERRUPTIBLE && signal_pending(prev))
        prev->state = TASK_RUNNING;
    else {
        if (prev->state == TASK_UNINTERRUPTIBLE)
            rq->nr_uninterruptible++;
        deactivate_task(prev, rq);
    }
}
```

The deactivate_task( ) function removes the process from the runqueue:

```
rq->nr_running--;
dequeue_task(p, p->array);
p->array = NULL;
```

Now, schedule( ) checks the number of runnable processes left in the runqueue. If there are some runnable processes, the function invokes the dependent_sleeper( ) function. In most cases, this function immediately returns zero. If, however, the kernel supports the hyper-threading technology (see the section "Runqueue Balancing in Multiprocessor Systems" later in this chapter), the function checks whether the process that is going to be selected for execution has significantly lower priority than a sibling process already running on a logical CPU of the same physical CPU; in this particular case, schedule( ) refuses to select the lower privilege process and executes the swapper process instead.

```
if (rq->nr_running) {
    if (dependent_sleeper(smp_processor_id( ), rq)) {
        next = rq->idle;
        goto switch_tasks;
    }
}
```

If no runnable process exists, the function invokes `idle_balance( )` to move some runnable process from another runqueue to the local runqueue; `idle_balance( )` is similar to `load_balance( )`, which is described in the later section "The load_balance( ) Function."

```
if (!rq->nr_running) {
    idle_balance(smp_processor_id( ), rq);
    if (!rq->nr_running) {
        next = rq->idle;
        rq->expired_timestamp = 0;
        wake_sleeping_dependent(smp_processor_id( ), rq);
        if (!rq->nr_running)
            goto switch_tasks;
    }
}
```

If `idle_balance( )` fails in moving some process in the local runqueue, `schedule( )` invokes `wake_sleeping_dependent( )` to reschedule runnable processes in idle CPUs (that is, in every CPU that runs the swapper process). As explained earlier when discussing the `dependent_sleeper( )` function, this unusual case might happen when the kernel supports the hyper-threading technology. However, in uniprocessor systems, or when all attempts to move a runnable process in the local runqueue have failed, the function picks the swapper process as `next` and continues with the next phase.

Let's suppose that the `schedule( )` function has determined that the runqueue includes some runnable processes; now it has to check that at least one of these runnable processes is active. If not, the function exchanges the contents of the `active` and `expired` fields of the runqueue data structure; thus, all expired processes become active, while the empty set is ready to receive the processes that will expire in the future.

```
array = rq->active;
if (!array->nr_active) {
    rq->active = rq->expired;
    rq->expired = array;
    array = rq->active;
    rq->expired_timestamp = 0;
    rq->best_expired_prio = 140;
}
```

It is time to look up a runnable process in the active `prio_array_t` data structure (see the section "Identifying a Process" in Chapter 3). First of all, `schedule( )` searches for the first nonzero bit in the bitmask of the active set. Remember that a bit in the bitmask is set when the corresponding priority list is not empty. Thus, the index of the first nonzero bit indicates the list containing the best

process to run. Then, the first process descriptor in that list is retrieved:

```
idx = sched_find_first_bit(array->bitmap);
next = list_entry(array->queue[idx].next, task_t, run_list);
```

The `sched_find_first_bit( )` function is based on the `bsfl` assembly language instruction, which returns the bit index of the least significant bit set to one in a 32-bit word.

The `next` local variable now stores the descriptor pointer of the process that will replace `prev`. The `schedule( )` function looks at the `next->activated` field. This field encodes the state of the process when it was awakened, as illustrated in Table 7-6.

Table 7-6. The meaning of the activated field in the process descriptor

Value	Description
0	The process was in `TASK_RUNNING` state.
1	The process was in `TASK_INTERRUPTIBLE` or `TASK_STOPPED` state, and it is being awakened by a system call service routine or a kernel thread.
2	The process was in `TASK_INTERRUPTIBLE` or `TASK_STOPPED` state, and it is being awakened by an interrupt handler or a deferrable function.
-1	The process was in `TASK_UNINTERRUPTIBLE` state and it is being awakened.

If `next` is a conventional process and it is being awakened from the `TASK_INTERRUPTIBLE` or `TASK_STOPPED` state, the scheduler adds to the average sleep time of the process the nanoseconds elapsed since the process was inserted into the runqueue. In other words, the sleep time of the process is increased to cover also the time spent by the process in the runqueue waiting for the CPU:

```
if (next->prio >= 100 && next->activated > 0) {
    unsigned long long delta = now - next->timestamp;
    if (next->activated == 1)
        delta = (delta * 38) / 128;
    array = next->array;
    dequeue_task(next, array);
    recalc_task_prio(next, next->timestamp + delta);
    enqueue_task(next, array);
}
next->activated = 0;
```

Observe that the scheduler makes a distinction between a process awakened by an interrupt handler or deferrable function, and a process awakened by a system call service routine or a kernel thread. In the former case, the scheduler adds the whole runqueue waiting time, while in the latter it adds just a fraction of that time. This is because interactive processes are more likely to be awakened by asynchronous events (think of the user pressing keys on the keyboard) rather than by synchronous ones.

7.4.4.4. Actions performed by schedule( ) to make the process switch

Now the `schedule( )` function has determined the next process to run. In a moment, the kernel will access the `tHRead_info` data structure of `next`, whose address is stored close to the top of `next`'s process descriptor:

```
switch_tasks:

prefetch(next);
```

The `prefetch` macro is a hint to the CPU control unit to bring the contents of the first fields of `next`'s process descriptor in the hardware cache. It is here just to improve the performance of `schedule( )`, because the data are moved in parallel to the execution of the following instructions, which do not affect `next`.

Before replacing `prev`, the scheduler should do some administrative work:

```
clear_tsk_need_resched(prev);
rcu_qsctr_inc(prev->thread_info->cpu);
```

The `clear_tsk_need_resched( )` function clears the `TIF_NEED_RESCHED` flag of `prev`, just in case `schedule( )` has been invoked in the lazy way. Then, the function records that the CPU is going through a quiescent state (see the section "Read-Copy Update (RCU)" in Chapter 5).

The `schedule( )` function must also decrease the average sleep time of `prev`, charging to it the slice of CPU time used by the process:

```
prev->sleep_avg -= run_time;
if ((long)prev->sleep_avg <= 0)
    prev->sleep_avg = 0;
prev->timestamp = prev->last_ran = now;
```

The timestamps of the process are then updated.

It is quite possible that `prev` and `next` are the same process: this happens if no other higher or equal priority active process is present in the runqueue. In this case, the function skips the process switch:

```
if (prev == next) {
    spin_unlock_irq(&rq->lock);
    goto finish_schedule;
}
```

At this point, prev and next are different processes, and the process switch is for real:

```
next->timestamp = now;
rq->nr_switches++;
rq->curr = next;
prev = context_switch(rq, prev, next);
```

The context_switch( ) function sets up the address space of next. As we'll see in "Memory Descriptor of Kernel Threads" in Chapter 9, the active_mm field of the process descriptor points to the memory descriptor that is used by the process, while the mm field points to the memory descriptor owned by the process. For normal processes, the two fields hold the same address; however, a kernel thread does not have its own address space and its mm field is always set to NULL. The context_switch( ) function ensures that if next is a kernel thread, it uses the address space used by prev:

```
if (!next->mm) {
    next->active_mm = prev->active_mm;
    atomic_inc(&prev->active_mm->mm_count);
    enter_lazy_tlb(prev->active_mm, next);
}
```

Up to Linux version 2.2, kernel threads had their own address space. That design choice was suboptimal, because the Page Tables had to be changed whenever the scheduler selected a new process, even if it was a kernel thread. Because kernel threads run in Kernel Mode, they use only the fourth gigabyte of the linear address space, whose mapping is the same for all processes in the system. Even worse, writing into the cr3 register invalidates all TLB entries (see "Translation Lookaside Buffers (TLB)" in Chapter 2), which leads to a significant performance penalty. Linux is nowadays much more efficient because Page Tables aren't touched at all if next is a kernel thread. As further optimization, if next is a kernel thread, the schedule( ) function sets the process into lazy TLB mode (see the section "Translation Lookaside Buffers (TLB)" in Chapter 2).

Conversely, if next is a regular process, the context_switch( ) function replaces the address space of prev with the one of next:

```
if (next->mm)
    switch_mm(prev->active_mm, next->mm, next);
```

If prev is a kernel thread or an exiting process, the context_switch( ) function saves the pointer to the memory descriptor used by prev in the runqueue's prev_mm field, then resets prev->active_mm:

```
if (!prev->mm) {
    rq->prev_mm = prev->active_mm;
    prev->active_mm = NULL;
}
```

Now context_switch( ) can finally invoke switch_to( ) to perform the process switch between prev and next (see the section "Performing the Process Switch" in Chapter 3):

```
switch_to(prev, next, prev);
```

```
return prev;
```

7.4.4.5. Actions performed by schedule( ) after a process switch

The instructions of the `context_switch( )` and `schedule( )` functions following the `switch_to` macro invocation will not be performed right away by the `next` process, but at a later time by `prev` when the scheduler selects it again for execution. However, at that moment, the `prev` local variable does not point to our original process that was to be replaced when we started the description of `schedule( )`, but rather to the process that was replaced by our original `prev` when it was scheduled again. (If you are confused, go back and read the section "Performing the Process Switch" in Chapter 3.) The first instructions after a process switch are:

```
barrier( );
finish_task_switch(prev);
```

Right after the invocation of the `context_switch( )` function in `schedule( )`, the `barrier( )` macro yields an optimization barrier for the code (see the section "Optimization and Memory Barriers" in Chapter 5). Then, the `finish_task_switch( )` function is executed:

```
mm = this_rq( )->prev_mm;
this_rq( )->prev_mm = NULL;
prev_task_flags = prev->flags;
spin_unlock_irq(&this_rq( )->lock);
if (mm)
    mmdrop(mm);
if (prev_task_flags & PF_DEAD)
    put_task_struct(prev);
```

If `prev` is a kernel thread, the `prev_mm` field of the runqueue stores the address of the memory descriptor that was lent to `prev`. As we'll see in Chapter 9, `mmdrop( )` decreases the usage counter of the memory descriptor; if the counter reaches 0 (likely because `prev` is a zombie process), the function also frees the descriptor together with the associated Page Tables and virtual memory regions.

The `finish_task_switch( )` function also releases the spin lock of the runqueue and enables the local interrupts. Then, it checks whether `prev` is a zombie task that is being removed from the system (see the section "Process Termination" in Chapter 3); if so, it invokes `put_task_struct( )` to free the process descriptor reference counter and drop all remaining references to the process (see the section "Process Removal" in Chapter 3).

The very last instructions of the `schedule( )` function are:

```
finish_schedule:

prev = current;
if (prev->lock_depth >= 0)
    _ _reacquire_kernel_lock( );
preempt_enable_no_resched();
if (test_bit(TIF_NEED_RESCHED, &current_thread_info( )->flags)
   goto need_resched;
return;
```

As you see, `schedule( )` reacquires the big kernel lock if necessary, reenables kernel preemption, and checks whether some other process has set the `TIF_NEED_RESCHED` flag of the current process. In this case, the whole `schedule( )` function is reexecuted from the beginning; otherwise, the function terminates.

7.5. Runqueue Balancing in Multiprocessor Systems

We have seen in Chapter 4 that Linux sticks to the Symmetric Multiprocessing model (SMP); this means, essentially, that the kernel should not have any bias toward one CPU with respect to the others. However, multiprocessor machines come in many different flavors, and the scheduler behaves differently depending on the hardware characteristics. In particular, we will consider the following three types of multiprocessor machines:

Classic multiprocessor architecture
: Until recently, this was the most common architecture for multiprocessor machines. These machines have a common set of RAM chips shared by all CPUs.

Hyper-threading
: A hyper-threaded chip is a microprocessor that executes several threads of execution at once; it includes several copies of the internal registers and quickly switches between them. This technology, which was invented by Intel, allows the processor to exploit the machine cycles to execute another thread while the current thread is stalled for a memory access. A hyper-threaded physical CPU is seen by Linux as several different logical CPUs.

NUMA
: CPUs and RAM chips are grouped in local "nodes" (usually a node includes one CPU and a few RAM chips). The memory arbiter (a special circuit that serializes the accesses to RAM performed by the CPUs in the system, see the section "Memory Addresses" in Chapter 2) is a bottleneck for the performance of the classic multiprocessor systems. In a NUMA architecture, when a CPU accesses a "local" RAM chip inside its own node, there is little or no contention, thus the access is usually fast; on the other hand, accessing a "remote" RAM chip outside of its node is much slower. We'll mention in the section "Non-Uniform Memory Access (NUMA)" in Chapter 8 how the Linux kernel memory allocator supports NUMA architectures.

These basic kinds of multiprocessor systems are often combined. For instance, a motherboard that includes two different hyper-threaded CPUs is seen by the kernel as four logical CPUs.

As we have seen in the previous section, the `schedule( )` function picks the new process to run from the runqueue of the local CPU. Therefore, a given CPU can execute only the runnable processes that are contained in the corresponding runqueue. On the other hand, a runnable process is always stored in exactly one runqueue: no runnable process ever appears in two or more runqueues. Therefore, until a process remains runnable, it is usually bound to one CPU.

This design choice is usually beneficial for system performance, because the hardware cache of every CPU is likely to be filled with data owned by the runnable processes in the runqueue. In some cases, however, binding a runnable process to a given CPU might induce a severe performance penalty. For instance, consider a large number of batch processes that make heavy use of the CPU: if most of them end up in the same runqueue, one CPU in the system will be overloaded, while the others will be nearly idle.

Therefore, the kernel periodically checks whether the workloads of the runqueues are balanced and, if necessary, moves some process from one runqueue to another. However, to get the best performance from a

multiprocessor system, the load balancing algorithm should take into consideration the topology of the CPUs in the system. Starting from kernel version 2.6.7, Linux sports a sophisticated runqueue balancing algorithm based on the notion of "scheduling domains." Thanks to the scheduling domains, the algorithm can be easily tuned for all kinds of existing multiprocessor architectures (and even for recent architectures such as those based on the "multi-core" microprocessors).

7.5.1. Scheduling Domains

Essentially, a scheduling domain is a set of CPUs whose workloads should be kept balanced by the kernel. Generally speaking, scheduling domains are hierarchically organized: the top-most scheduling domain, which usually spans all CPUs in the system, includes children scheduling domains, each of which include a subset of the CPUs. Thanks to the hierarchy of scheduling domains, workload balancing can be done in a rather efficient way.

Every scheduling domain is partitioned, in turn, in one or more groups, each of which represents a subset of the CPUs of the scheduling domain. Workload balancing is always done between groups of a scheduling domain. In other words, a process is moved from one CPU to another only if the total workload of some group in some scheduling domain is significantly lower than the workload of another group in the same scheduling domain.

Figure 7-2 illustrates three examples of scheduling domain hierarchies, corresponding to the three main architectures of multiprocessor machines.

Figure 7-2. Three examples of scheduling domain hierarchies

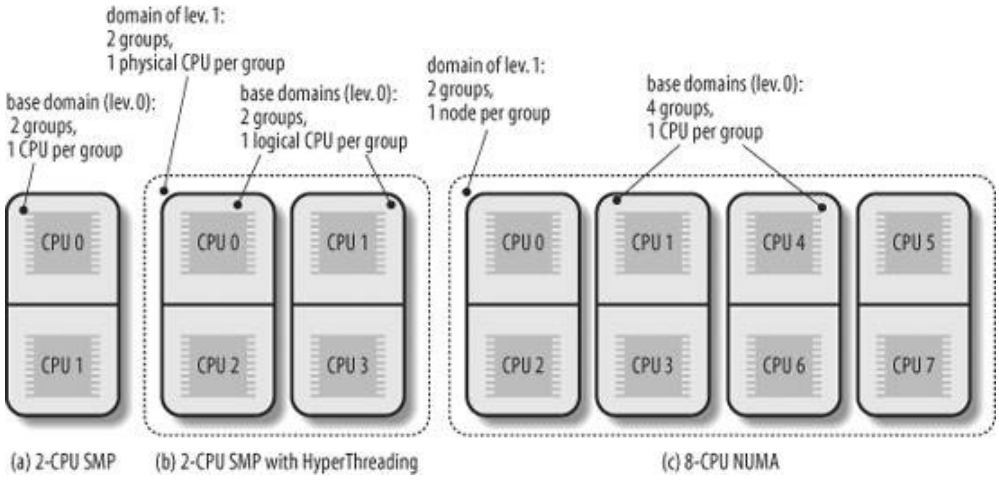


Figure 7-2 (a) represents a hierarchy composed by a single scheduling domain for a 2-CPU classic multiprocessor architecture. The scheduling domain includes only two groups, each of which includes one CPU.

Figure 7-2 (b) represents a two-level hierarchy for a 2-CPU multiprocessor box with hyper-threading technology. The top-level scheduling domain spans all four logical CPUs in the system, and it is composed by two groups. Each group of the top-level domain corresponds to a child scheduling domain and spans a physical CPU. The bottom-level scheduling domains (also called base scheduling domains) include two groups, one for each logical CPU.

Finally, Figure 7-2 (c) represents a two-level hierarchy for an 8-CPU NUMA architecture with two nodes and four CPUs per node. The top-level domain is organized in two groups, each of which corresponds to a different node. Every base scheduling domain spans the CPUs inside a single node and has four groups, each of which spans a single CPU.

Every scheduling domain is represented by a `sched_domain` descriptor, while every group inside a scheduling domain is represented by a `sched_group` descriptor. Each `sched_domain` descriptor includes a field `groups`, which points to the first element in a list of group descriptors. Moreover, the `parent` field of the `sched_domain` structure points to the descriptor of the parent scheduling domain, if any.

The `sched_domain` descriptors of all physical CPUs in the system are stored in the per-CPU variable `phys_domains`. If the kernel does not support the hyper-threading technology, these domains are at the bottom level of the domain hierarchy, and the `sd` fields of the runqueue descriptors point to themthat is, they are the base scheduling domains. Conversely, if the kernel supports the hyper-threading technology, the bottom-level scheduling domains are stored in the per-CPU variable `cpu_domains`.

7.5.2. The rebalance_tick( ) Function

To keep the runqueues in the system balanced, the `rebalance_tick( )` function is invoked by `scheduler_tick( )` once every tick. It receives as its parameters the index `this_cpu` of the local CPU, the address `this_rq` of the local runqueue, and a flag, `idle`, which can assume the following values:

SCHED_IDLE
: The CPU is currently idle, that is, `current` is the swapper process.

NOT_IDLE
: The CPU is not currently idle, that is, `current` is not the swapper process.

The `rebalance_tick( )` function determines first the number of processes in the runqueue and updates the runqueue's average workload; to do this, the function accesses the `nr_running` and `cpu_load` fields of the runqueue descriptor.

Then, `rebalance_tick( )` starts a loop over all scheduling domains in the path from the base domain (referenced by the `sd` field of the local runqueue descriptor) to the top-level domain. In each iteration the function determines whether the time has come to invoke the `load_balance( )` function, thus executing a rebalancing operation on the scheduling domain. The value of `idle` and some parameters stored in the `sched_domain` descriptor determine the frequency of the invocations of `load_balance( )`. If `idle` is equal to `SCHED_IDLE`, then the runqueue is empty, and `rebalance_tick( )` invokes `load_balance( )` quite often (roughly once every one or two ticks for scheduling domains corresponding to logical and physical CPUs). Conversely, if `idle` is equal to `NOT_IDLE`, `rebalance_tick( )` invokes `load_balance( )` sparingly (roughly once every 10 milliseconds for scheduling domains corresponding to logical CPUs, and once every 100 milliseconds for scheduling domains corresponding to physical CPUs).

7.5.3. The load_balance( ) Function

The load_balance( ) function checks whether a scheduling domain is significantly unbalanced; more precisely, it checks whether unbalancing can be reduced by moving some processes from the busiest group to the runqueue of the local CPU. If so, the function attempts this migration. It receives four parameters:

this_cpu
: The index of the local CPU

this_rq
: The address of the descriptor of the local runqueue

sd
: Points to the descriptor of the scheduling domain to be checked

idle
: Either SCHED_IDLE (local CPU is idle) or NOT_IDLE

The function performs the following operations:

1. Acquires the this_rq->lock spin lock.
2. Invokes the find_busiest_group( ) function to analyze the workloads of the groups inside the scheduling domain. The function returns the address of the sched_group descriptor of the busiest group, provided that this group does not include the local CPU; in this case, the function also returns the number of processes to be moved into the local runqueue to restore balancing. On the other hand, if either the busiest group includes the local CPU or all groups are essentially balanced, the function returns NULL. This procedure is not trivial, because the function tries to filter the statistical fluctuations in the workloads.
3. If find_busiest_group( ) did not find a group not including the local CPU that is significantly busier than the other groups in the scheduling domain, the function releases the this_rq->lock spin lock, tunes the parameters in the scheduling domain descriptor so as to delay the next invocation of load_balance( ) on the local CPU, and terminates.
4. Invokes the find_busiest_queue( ) function to find the busiest CPUs in the group found in step 2. The function returns the descriptor address busiest of the corresponding runqueue.
5. Acquires a second spin lock, namely the busiest->lock spin lock. To prevent deadlocks, this has to be done carefully: the this_rq->lock is first released, then the two locks are acquired by increasing CPU indices.
6. Invokes the move_tasks( ) function to try moving some processes from the busiest runqueue to the local runqueue this_rq (see the next section).
7. If the move_task( ) function failed in migrating some process to the local runqueue, the scheduling domain is still unbalanced. Sets to 1 the busiest->active_balance flag and wakes up the migration kernel thread whose descriptor is stored in busiest->migration_thread. The migration kernel thread walks the chain of the scheduling domain, from the base domain of the busiest runqueue to the top domain, looking for an idle CPU. If an idle CPU is found, the kernel thread invokes move_tasks( ) to move one process into the idle runqueue.
8. Releases the busiest->lock and this_rq->lock spin locks.
9. Terminates.

7.5.4. The move_tasks( ) Function

The `move_tasks( )` function moves processes from a source runqueue to the local runqueue. It receives six parameters: `this_rq` and `this_cpu` (the local runqueue descriptor and the local CPU index), `busiest` (the source runqueue descriptor), `max_nr_move` (the maximum number of processes to be moved), `sd` (the address of the scheduling domain descriptor in which this balancing operation is carried on), and the `idle` flag (beside `SCHED_IDLE` and `NOT_IDLE`, this flag can also be set to `NEWLY_IDLE` when the function is indirectly invoked by `idle_balance( )`; see the section "The schedule( ) Function" earlier in this chapter).

The function first analyzes the expired processes of the `busiest` runqueue, starting from the higher priority ones. When all expired processes have been scanned, the function scans the active processes of the `busiest` runqueue. For each candidate process, the function invokes `can_migrate_task( )`, which returns 1 if all the following conditions hold:

- The process is not being currently executed by the remote CPU.
- The local CPU is included in the `cpus_allowed` bitmask of the process descriptor.
- At least one of the following holds:
  - The local CPU is idle. If the kernel supports the hyper-threading technology, all logical CPUs in the local physical chip must be idle.
  - The kernel is having trouble in balancing the scheduling domain, because repeated attempts to move processes have failed.
  - The process to be moved is not "cache hot" (it has not recently executed on the remote CPU, so one can assume that no data of the process is included in the hardware cache of the remote CPU).

If `can_migrate_task( )` returns the value 1, `move_tasks( )` invokes the `pull_task( )` function to move the candidate process to the local runqueue. Essentially, `pull_task( )` executes `dequeue_task( )` to remove the process from the remote runqueue, then executes `enqueue_task( )` to insert the process in the local runqueue, and finally, if the process just moved has higher dynamic priority than `current`, invokes `resched_task( )` to preempt the current process of the local CPU.

7.6. System Calls Related to Scheduling

Several system calls have been introduced to allow processes to change their priorities and scheduling policies. As a general rule, users are always allowed to lower the priorities of their processes. However, if they want to modify the priorities of processes belonging to some other user or if they want to increase the priorities of their own processes, they must have superuser privileges.

7.6.1. The nice( ) System Call

The `nice( )` [*] system call allows processes to change their base priority. The integer value contained in the `increment` parameter is used to modify the `nice` field of the process descriptor. The *nice* Unix command, which allows users to run programs with modified scheduling priority, is based on this system call.

> [*] Because this system call is usually invoked to lower the priority of a process, users who invoke it for their processes are "nice" to other users.

The `sys_nice( )` service routine handles the `nice( )` system call. Although the `increment` parameter may have any value, absolute values larger than 40 are trimmed down to 40. Traditionally, negative values correspond to requests for priority increments and require superuser privileges, while positive ones correspond to requests for priority decreases. In the case of a negative increment, the function invokes the `capable( )` function to verify whether the process has a `CAP_SYS_NICE` capability. Moreover, the function invokes the `security_task_setnice( )` security hook. We discuss that function in Chapter 20. If the user turns out to have the privilege required to change priorities, `sys_nice( )` converts `current->static_prio` to the range of nice values, adds the value of `increment`, and invokes the `set_user_nice( )` function. In turn, the latter function gets the local runqueue lock, updates the static priority of `current`, invokes the `resched_task( )` function to allow other processes to preempt `current`, and release the runqueue lock.

The `nice( )` system call is maintained for backward compatibility only; it has been replaced by the `setpriority( )` system call described next.

7.6.2. The getpriority( ) and setpriority( ) System Calls

The `nice( )` system call affects only the process that invokes it. Two other system calls, denoted as `getpriority( )` and `setpriority( )`, act on the base priorities of all processes in a given group. `getpriority( )` returns 20 minus the lowest `nice` field value among all processes in a given groupthat is, the highest priority among those processes; `setpriority( )` sets the base priority of all processes in a given group to a given value.

The kernel implements these system calls by means of the `sys_getpriority( )` and `sys_setpriority( )` service routines. Both of them act essentially on the same group of parameters:

which
: The value that identifies the group of processes; it can assume one of the following:

PRIO_PROCESS
: Selects the processes according to their process ID (`pid` field of the process descriptor).

PRIO_PGRP
: Selects the processes according to their group ID (`pgrp` field of the process descriptor).

PRIO_USER
: Selects the processes according to their user ID (`uid` field of the process descriptor).

who
: The value of the `pid`, `pgrp`, or `uid` field (depending on the value of `which`) to be used for selecting the processes. If `who` is 0, its value is set to that of the corresponding field of the `current` process.

niceval
: The new base priority value (needed only by `sys_setpriority( )`). It should range between -20 (highest priority) and + 19 (lowest priority).

As stated before, only processes with a `CAP_SYS_NICE` capability are allowed to increase their own base priority or to modify that of other processes.

As we will see in Chapter 10, system calls return a negative value only if some error occurred. For this reason, `getpriority( )` does not return a normal nice value ranging between - 20 and + 19, but rather a nonnegative value ranging between 1 and 40.

7.6.3. The sched_getaffinity( ) and sched_setaffinity( ) System Calls

The `sched_getaffinity( )` and `sched_setaffinity( )` system calls respectively return and set up the CPU affinity mask of a processthe bit mask of the CPUs that are allowed to execute the process. This mask is stored in the `cpus_allowed` field of the process descriptor.

The `sys_sched_getaffinity( )` system call service routine looks up the process descriptor by invoking `find_task_by_pid( )`, and then returns the value of the corresponding `cpus_allowed` field ANDed with the bitmap of the available CPUs.

The `sys_sched_setaffinity( )` system call is a bit more complicated. Besides looking for the descriptor of the target process and updating the `cpus_allowed` field, this function has to check whether the process is included in a runqueue of a CPU that is no longer present in the new affinity mask. In the worst case, the process has to be moved from one runqueue to another one. To avoid problems due to deadlocks and race conditions, this job is done by the migration kernel threads (there is one thread per CPU). Whenever a process has to be moved from a runqueue `rq1` to another runqueue `rq2`, the system call awakes the migration thread of `rq1` (`rq1->migration_thread`), which in turn removes the process from `rq1` and inserts it into `rq2`.

7.6.4. System Calls Related to Real-Time Processes

We now introduce a group of system calls that allow processes to change their scheduling discipline and, in particular, to become real-time processes. As usual, a process must have a `CAP_SYS_NICE` capability to modify the values of the `rt_priority` and `policy` process descriptor fields of any process, including itself.

7.6.4.1. The sched_getscheduler( ) and sched_setscheduler( ) system calls

The `sched_getscheduler( )` system call queries the scheduling policy currently applied to the process identified by the `pid` parameter. If `pid` equals 0, the policy of the calling process is retrieved. On success, the system call returns the policy for the process: `SCHED_FIFO`, `SCHED_RR`, or `SCHED_NORMAL` (the latter is also called `SCHED_OTHER`). The corresponding `sys_sched_getscheduler( )` service routine invokes `find_process_by_pid( )`, which locates the process descriptor corresponding to the given `pid` and returns the value of its `policy` field.

The `sched_setscheduler( )` system call sets both the scheduling policy and the associated parameters for the process identified by the parameter `pid`. If `pid` is equal to 0, the scheduler parameters of the calling process will be set.

The corresponding `sys_sched_setscheduler( )` system call service routine simply invokes `do_sched_setscheduler( )`. The latter function checks whether the scheduling policy specified by the `policy` parameter and the new priority specified by the `param->sched_priority` parameter are valid. It also checks whether the process has `CAP_SYS_NICE` capability or whether its owner has superuser rights. If everything is OK, it removes the process from its runqueue (if it is runnable); updates the static, real-time, and dynamic priorities of the process; inserts the process back in the runqueue; and finally invokes, if necessary, the `resched_task( )` function to preempt the current process of the runqueue.

7.6.4.2. The sched_ getparam( ) and sched_setparam( ) system calls

The `sched_getparam( )` system call retrieves the scheduling parameters for the process identified by `pid`. If `pid` is 0, the parameters of the `current` process are retrieved. The corresponding `sys_sched_getparam( )` service routine, as one would expect, finds the process descriptor pointer associated with `pid`, stores its `rt_priority` field in a local variable of type `sched_param`, and invokes `copy_to_user( )` to copy it into the process address space at the address specified by the `param` parameter.

The `sched_setparam( )` system call is similar to `sched_setscheduler( )`. The difference is that `sched_setparam( )` does not let the caller set the `policy` field's value.[*] The corresponding `sys_sched_setparam( )` service routine invokes `do_sched_setscheduler( )`, with almost the same parameters as `sys_sched_setscheduler( )`.

[*] This anomaly is caused by a specific requirement of the POSIX standard.

7.6.4.3. The sched_ yield( ) system call

The `sched_yield( )` system call allows a process to relinquish the CPU voluntarily without being suspended; the process remains in a `TASK_RUNNING` state, but the scheduler puts it either in the expired set

of the runqueue (if the process is a conventional one), or at the end of the runqueue list (if the process is a real-time one). The `schedule( )` function is then invoked. In this way, other processes that have the same dynamic priority have a chance to run. The call is used mainly by `SCHED_FIFO` real-time processes.

7.6.4.4. The sched_ get_priority_min( ) and sched_ get_priority_max( ) system calls

The `sched_get_priority_min( )` and `sched_get_priority_max( )` system calls return, respectively, the minimum and the maximum real-time static priority value that can be used with the scheduling policy identified by the `policy` parameter.

The `sys_sched_get_priority_min( )` service routine returns 1 if `current` is a real-time process, 0 otherwise.

The `sys_sched_get_priority_max( )` service routine returns 99 (the highest priority) if `current` is a real-time process, 0 otherwise.

7.6.4.5. The sched_rr_ get_interval( ) system call

The `sched_rr_get_interval( )` system call writes into a structure stored in the User Mode address space the Round Robin time quantum for the real-time process identified by the `pid` parameter. If `pid` is zero, the system call writes the time quantum of the current process.

The corresponding `sys_sched_rr_get_interval( )` service routine invokes, as usual, `find_process_by_pid( )` to retrieve the process descriptor associated with `pid`. It then converts the base time quantum of the selected process into seconds and nanoseconds and copies the numbers into the User Mode structure. Conventionally, the time quantum of a FIFO real-time process is equal to zero.